

Проектные задачи по предмету «**Методы машинного обучения**».

Команда: 4–6 человек.

Итеративность: месячные спринты, с промежуточными отчетами в конце октября и ноября. Итоговый отчет в декабре.

Прозрачность: вся работа ведётся публично для преподавателя: доска задач, репозиторий, документы и отчёты.

Общие требования:

1. Роли и распределение обязанностей

Участники команды должны распределить обязанности таким образом, чтобы каждый член команды вносил вклад в проект. Это включает в себя определение ролей и ответственностей. Таким образом у каждого студента должна быть определена и зафиксирована роль.

Базовые роли (обязательны):

- Менеджер проекта: спринт-планирование, доска, отчётность, риски.
- Разработчики / ML-инженеры: код, фичи, модели, пайплайны, MLOps-настройки.
- Тестировщик / QA (ML-QA): стратегия тестов, качество данных и кода, контроль метрик и воспроизводимости.

Критерий приёмки: у каждого студента зафиксирована роль и зона ответственности; в репозитории есть файл TEAM.md

2. Управление проектом

- Каждая команда должна использовать инструмент управления проектами, для визуализации, отслеживания и управления задачами («Scrum-лайт» или Канбан).
- Задачи должны быть созданы, привязаны к определенным участникам команды, иметь определенные сроки выполнения и отображаться на доске проекта. Бэклог с приоритизацией (ценность/риск/зависимости). Планирование спринта: цели спринта и критерии «готово». Реестр рисков: вероятность × влияние, план реагирования.

Критерий приёмки: у задач есть владелец, сроки, статусы; история изменений видна; прослеживается связь задач с коммитами/релизами. Ссылка на доску + скриншоты состояний в контрольные точки.

3. Контроль версий и отчётность по Git

- Обязательным является использование системы контроля версий. Создать свой репозиторий для проекта на одной из платформ
- Коммиты: хранить и отслеживать записи изменений в репозитории. Каждый коммит обычно сопровождается комментарием, описывающим внесенные изменения. Регулярное создание коммитов после завершения конкретной задачи или добавления новой функциональности. Защищённая главная ветка, ревью перед слиянием, 1–2 независимых ревьюера.

Критерий приёмки: репозиторий создан, коммиты регулярные, фичи/фиксы — через ветки и PR; видна роль ответственного за слияние. **Публичный или доступный преподавателю репозиторий.** Пакет «снимков» активности: граф коммитов, список PR, релизы, участники.

4. Документация (doc-as-code):

Все этапы проекта, начиная от планирования и заканчивая реализацией, должны быть документированы.

Структура в репозитории (/docs):

- README.md (как запустить/использовать/структура проекта).
- ARCHITECTURE.md (диаграмма потоков данных/пайплайнов, ключевые решения).
- DATA_README.md (источники, словарь, схемы, ограничения, лицензии).
- TESTING.md (стратегия тестов и покрытие).

ML-специфика:

- EXPERIMENTS.md (сетка экспериментов, протоколы, ссылки на артефакты).
- MODEL_CARD.md (цель, данные, метрики, ограничения, риски применения).
- EVALUATION.md (валидация/кросс-валидация, процедура отбора финальной модели).

Критерий приёмки: есть README, описаны методы/инструменты/результаты; документация актуальна и соответствует реализованной системе.

5. Тестирование и оценка адекватности решений

В командах должна быть отведена роль ответственного за тестирование продукта и оценку качества кода и решений.

Пирамида тестов:

- **Quality of Data:** схемы/диапазоны/уникальность/свежесть; детект утечек таргета.
- Unit-тесты ключевой логики (цель: $\geq 70\%$).
- **Интеграционные тесты:** сборка пайплайна (fit→predict), сохранение/загрузка артефактов End-to-End (минимум один «сквозной» сценарий).
- **End-to-End тест:** минимальный сценарий от «сырых» данных до метрик/инференса.
- Тесты воспроизводимости: фиксированные сиды, детерминированные пайплайны, контроль версий данных/моделей.
- Тесты адекватности: сравнение с бейслайном (напр., «предсказываем среднее/частоту/популярность»), проверка переобучения (gap train/val), калибровка вероятностей (для классификации), стабильность на бэкteste (для временных рядов).

Критерий приёмки: есть план тестирования, авто-запуски, отчёты, и «красные флаги» на нарушениях качества данных/порогов метрик.

6. Презентация результатов

Формат: 10–12 минут + 3–5 минут Q&A.

Содержание:

- проблема и критерии успеха;
- архитектура и ключевые решения;
- демонстрация;
- метрики качества и тесты;
- риски/ограничения;
- уроки;
- планы развития.

Сдача: репозиторий, живое демо/видео, слайды (PDF) + ссылки на доску задач и артефакты.

7. Рекомендуемые современные практики (обогащение)

- Reproducible dev-env: requirements.txt/pyproject.toml, контейнеризация; .env/секреты.
- Эксперимент-трекинг: любой инструмент (MLflow/W&B/DVC Experiments) — без привязки к бренду; зафиксированные артефакты и параметры.
- Регистры: версионирование датасетов/моделей; data lineage.

Проектная задача 1.

Регрессия: прогноз стоимости жилья (Ames Housing)

Цель проекта. Построить воспроизводимый ML-пайплайн для регрессии по табличным данным: EDA → фичи → модель(и) → кросс-валидация → интерпретируемость → отчёт.

Источники данных.

- *House Prices — Advanced Regression Techniques* (Kaggle). [Kaggle](#)

Требования к реализации.

1. **EDA/качество:** пропуски, выбросы, утечка таргета; кодирование категорий; обработка редких категорий.
2. **Бейслайн:** среднее по группе/линейная регрессия с минимальными фичами.
3. **Модели:** регуляризованные (Ridge/Lasso/ElasticNet) и градиентный бустинг; сборка через Pipeline; Feature importance/SHAP.
4. **Валидация:** K-fold/StratifiedKFold (по квантилям таргета); подбор гиперпараметров с повторной валидацией.
5. **Метрики:** RMSE (основная), MAE; анализ ошибок (классы домов/квартир с низкой точностью).
6. **Репродуктивность:** фиксированные сиды; сохранение артефактов (модель/препроцессор).
7. **Артефакты:** ноутбуки/скрипты, MODEL_CARD.md, EXPERIMENTS.md.

Минимальные метрики/результаты. Кросс-валидированный RMSE, таблица сравнения моделей, топ-N ключевых признаков (SHAP), разбор ошибок в 3–5 сегментах.

Проектная задача 2.

Классификация: предсказание оттока клиентов телеком-компаний

Цель проекта. Разработать модель бинарной классификации с учётом дисбаланса классов и калибровкой вероятностей.

Источники данных.

- *Telco Customer Churn* (Kaggle, IBM). [Kaggle](#)

Требования к реализации.

1. **Подготовка:** кодирование категорий, нормализация числовых; анализ дисбаланса, базовый undersampling/oversampling (в разумных пределах).
2. **Бейслайны:** majority class / логистическая регрессия.
3. **Модели:** деревья/ансамбли (RandomForest/Gradient Boosting) + интерпретация (SHAP/feature importance).
4. **Валидация и калибровка:** стратифицированная K-fold; калибровка (Platt/изотоническая).
5. **Метрики:** ROC-AUC (основная), PR-AUC, F1 на оптимальном пороге; матрица ошибок, стоимость ошибок (пропишите бизнес-порог).

6. **Отчёт:** лидеры факторов риска, рекомендации удержания (по сегментам).

Минимальные метрики/результаты. $ROC-AUC \geq 0.75$ на валидации (или обоснование достигнутого уровня); таблица trade-off «чувствительность-точность» для разных порогов и сценариев.

Проектная задача 3.

Кластеризация: сегментация клиентов онлайн-ритейла (RFM + поведение)

Цель проекта. Построить сегментацию без учителя: выделить группы клиентов и описать их профили для CRM/маркетинга.

Источники данных.

- *Online Retail II (UCI)* на Kaggle. [Kaggle](#)
- Оригинальное описание набора в UCI ML Repository. [UCI Machine Learning Repository](#)

Требования к реализации.

1. **Очистка:** исключить возвраты/аномалии; агрегировать по клиенту.
2. RFM-фичи + поведенческие признаки (сезонность, ассортимент, чек). Масштабирование/стандартизация.
3. **Методы:** K-means / GMM / иерархическая кластеризация; выбор K по силуэту/Davies–Bouldin/Gap.
4. **Интерпретация:** профили сегментов (размер, ценность, частота, география); валидация стабильности сегментов на бутстрэпе.
5. **Витрина:** таблица dim_customer_segment + правила отнесения. Минимальные метрики/результаты. Сравнение 2–3 методов по внутренним метрикам; описания ≥ 4 сегментов с бизнес-интерпретацией и рекомендациями.

Проектная задача 4.

Рекомендательная система: персональные рекомендации фильмов (MovieLens)

Цель проекта. Сконструировать офлайн-оценённую рекомендательную систему с корректным временным сплитом и несколькими подходами.

Источники данных.

- *MovieLens 20M Dataset* (Kaggle). [Kaggle](#)

Требования к реализации.

1. **Сплиты по времени:** train до T, валидация/тест после T; негатив-сэмплинг.
2. **Бейслайны:** популярность/тренд/топ по жанру.
3. **Методы:** user-/item-based CF; матричная факторизация (ALS/SVD).

4. **Метрики:** Precision@K, Recall@K, MAP@K, NDCG@K (K выбрать и обосновать); coverage/diversity как дополнительные.
5. **Холодный старт:** простые правила/контентные признаки.
6. **Интерпретация и качество:** примеры топ-N рекомендаций для 3 профилей пользователей; разбор ошибок/тенденций. Минимальные метрики/результаты. Таблица офлайн-метрик по моделям; анализ влияния K и стратегий сплита; отчёт по cold-start.

Проектная задача 5.

Нейронные сети (CV): классификация изображений CIFAR-10 с transfer learning

Цель проекта. Собрать воспроизводимый DL-пайплайн: аугментации → fine-tuning предобученной сети → контроль переобучения и интерпретация.

Источники данных.

- *CIFAR-10* (Kaggle competition). [Kaggle](https://www.kaggle.com/c/cifar-10)

Требования к реализации.

1. **Подготовка:** train/val/test сплит; аугментации (flip/crop/brightness); нормализация.
 2. **Модели:** baseline-CNN и transfer learning (предобученная ResNet/MobileNet/ViT — на выбор). Early stopping, lr-scheduler.
 3. **Метрики:** Accuracy (основная), macro-F1, top-5 accuracy; confusion matrix; per-class отчёты.
 4. **Интерпретация:** Grad-CAM/сaliency карты (на 5–10 примерах) с разбором ошибок.
 5. **Ресурсы/воспроизводимость:** фиксированные сиды, контроль детерминизма; сохранение весов/логи обучения. Минимальные метрики/результаты. Сравнение baseline vs transfer (дельта по accuracy/F1); достигнутая accuracy $\geq$ базового бенчмарка; анализ ошибок для 3 сложных классов.
-